claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf_e3cc36d8-eca\agent-a30b899959f2531e9.jsonl`
- SHA-256: `266e20e4efd7edd314b698e42159dd95be95255de327ba432001bf9e71d1906c`
- Source modified: `2026-06-21T09:22:05+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_e3cc36d8-eca`
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`

Transcript

1. user (2026-06-21T09:19:21.058Z)

Engine repo at C:/Users/avidu/Projects/badminton-highlight-indexer. Your job: ADVERSARIALLY REFUTE this claim, do not confirm it.

CLAIM: "Under default single-user config, POST /api/process accepts a gs://|gcs:// video_path, fetches+processes it, and writes a video_id that db.get_video() returns and the SPA can load via ?video=."

YOUR LENS: WORKER/EXECUTION + DB WRITE: focus ONLY on the path after 202 ? backend/api/job_worker.py draining, _job_to_request rebuilding video_path, _execute_processing calling acquire_local_input, and whether add_video/play_segments actually persist so db.get_video(video_id) returns a row the SPA can query. Find any place the gs:// case diverges from the local case or fails to write the DB.

Read the actual code for your lens and try hard to find a CONCRETE counterexample (file:line) where a valid gs:// URI under default config fails to yield a queryable API-DB video_id. IMPORTANT: a failure of the real network fetch due to a MISSING dependency (google-cloud-storage) or MISSING auth (ADC) is ENVIRONMENTAL, not a code gate ? do NOT count those as a refutation; the claim is about code routing/acceptance, assuming the fetch itself can run. Only refute with a real code-path defect. Default refuted=true ONLY if you have a concrete counterexample; otherwise refuted=false. Return structured JSON.

2. assistant (2026-06-21T09:19:23.387Z)

I'll adversarially analyze the worker/execution + DB write path. Let me start by reading the relevant files.

3. user (2026-06-21T09:19:24.354Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10      `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11      `backend.main` module object at call-time (`import backend.main as _main`), never as a
12      bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13      ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14      `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
```

```
    inside the functions also avoids the circular import at module load (main.py imports
this
    module; this module must not import main at import time).
    """
    import logging
    import threading
    import traceback
    from concurrent.futures import ThreadPoolExecutor
    from typing import Any, Dict, Optional

    logger = logging.getLogger(__name__)

    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
    # already parallelize their AI handoff internally via their own pools.
    _job_executor: Optional[ThreadPoolExecutor] = None


    def _get_executor() -> ThreadPoolExecutor:
        """Lazy module-global executor; tests monkeypatch this for inline execution."""
        global _job_executor
        if _job_executor is None:
            _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
        return _job_executor


    class JobWaiting(Exception):
        """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
        WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
        NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
        releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
        is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.

        The wiring is additive: the production segmenter path never raises this today (zero
        behaviour change), so a job runs exactly as before. The hook is what a later slice
        (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
        """

        def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
            super().__init__(f"job parked for {delay_sec:.0f}s")
            self.delay_sec = max(0.0, float(delay_sec))
            self.progress = progress


    def _job_to_request(job: Dict[str, Any]):
        """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
        re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
        the original submit. The row stores exactly the ProcessRequest fields."""
        import backend.main as _main
        return _main.ProcessRequest(
            video_path=job["video_path"],
            player_pool=job.get("player_pool"),
            max_frames=job.get("max_frames"),
            segmenter=job.get("segmenter"),
        )


    def _run_claimed_job(job: Dict[str, Any]) -> None:
        """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
```

```
        terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).

        Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
        the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
        result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
        job self-scheduled and will be re-claimed when `next_poll_at` is due.
        """
        import backend.main as _main
        db_instance = _main.db_instance
        job_id = job["id"]
        req = _main._job_to_request(job)
        try:
            result = _main._execute_processing(
                req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
            if result.get("video_id"):
                db_instance.set_job_video_id(job_id, result["video_id"])
            db_instance.mark_job_done(job_id, result)
        except _main.JobWaiting as w:
            # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
            logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
            db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
        except Exception as e:
            logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
            db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


    def _drain_due_jobs() -> int:
        """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
        `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

        Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
        up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
        a single-worker box keeps making progress with no central timer (the DB is the
clock).
        Returns the number of jobs that reached a terminal/parked state this tick.
        """
        import backend.main as _main
        ran = 0
        while True:
            job = _main.db_instance.claim_due_job()
            if job is None:
                break
            ran += 1
            _main._run_claimed_job(job)
        # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
        # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
        # on the next submit/drain. Best-effort ? never raises into the worker.
        _main._schedule_next_drain_if_waiting()
        return ran


    def _schedule_next_drain_if_waiting() -> None:
        """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
        self-scheduled job resumes with no further client submit. The drain itself re-checks
        due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
        import backend.main as _main
```

```
129         try:
130             delay = _main.db_instance.seconds_until_next_due()
131         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
132             logger.error(f"Could not compute next-due delay: {e}")
133             return
134         if delay is None:
135             return  # nothing waiting
136         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
137         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
138 
139 
140     def _arm_wake_timer(delay_sec: float) -> None:
141         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
142         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
143         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
144         import backend.main as _main
145         timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
146         timer.daemon = True
147         timer.start()
148 
149 
150     #: A parked job further out than this is re-checked by a capped wake rather than one
long
151     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
152     _MAX_DRAIN_WAKE_SEC = 60.0
153 
```

4. user (2026-06-21T09:19:26.689Z)

```
Your configuration specifies to merge with the ref 'refs/heads/feat/serving-m4-device-context'
from the remote, but no such ref was fetched.
```

5. assistant (2026-06-21T09:19:30.610Z)

```
The git config references a missing ref but the local checkout is what matters. Let me trace the
execution path. I have `_job_to_request` and `_run_claimed_job`. Now I need
`_execute_processing` and `acquire_local_input`.
```

6. user (2026-06-21T09:19:32.148Z)

```
Found 6 files
backend\config\models.py
backend\main.py
backend\storage\__init__.py
backend\infrastructure\database.py
backend\api\job_worker.py
backend\utils\video.py
```

7. user (2026-06-21T09:19:33.414Z)

```
Found 15 files
backend\config\models.py
backend\utils\paths.py
backend\storage\__init__.py
backend\storage\ref.py
backend\config\presets.py
backend\worker\__main__.py
backend\utils\workspace.py
backend\storage\gdrive.py
backend\storage\base.py
```

```
backend\storage\s3.py
backend\storage\local.py
backend\storage\gcs.py
backend\annotations\automated_provider.py
backend\annotations\file_provider.py
backend\annotations\base.py
```

8. user (2026-06-21T09:19:37.014Z)

```
140:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
```

9. user (2026-06-21T09:19:38.068Z)

```
backend\config\models.py:314:    # scratch dir before processing (see
``backend.storage.acquire_local_input``).
backend\main.py:159:    local_video = storage.acquire_local_input(req.video_path,
getattr(global_config, "storage", None))
backend\main.py:607:        local_video = storage.acquire_local_input(args.video_path,
storage_cfg)
backend\storage\__init__.py:25:    "acquire_local_input", "input_is_local", "resolve_input_ref",
backend\storage\__init__.py:59:def acquire_local_input(raw: str, storage_config: Any = None, *,
```

10. user (2026-06-21T09:19:45.976Z)

```
140     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
141         """The full hash ? validate ? segment ? report pipeline for one video.
142
143         This is the former synchronous /api/process body, unchanged in behavior, now run on
144         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
145         the sync endpoint used to return (UI compatibility); the per-video observability
146         report is still emitted in `finally:` and grafted as response["reports"].
147
148         `progress` is an optional callable(stage: str) used to surface coarse job progress.
149         Raises on unexpected internal errors ? the caller records those as a job failure
150         (after this function has already restored the pre-run segments, see below).
151         """
152         notify = progress or (lambda stage: None)
153
154         notify("hashing")
155         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
156         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
157         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
158         # consumers (hash / validators / segmenter) use the resolved local path.
159         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
160         video_id = compute_video_id(local_video)
161         was_reingest = db_instance.get_video(video_id) is not None
162
163         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
164         notify("validating")
165         logger.info(f"Running validations for {req.video_path}...")
166         val_results, val_context = registry.run_all_with_context(local_video, global_config)
167         failures = [r for r in val_results if not r.passed]
168
169         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
170         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
171         seg_name = req.segmenter or default_seg
172         segmenter_actual = None
173         segmentation_ran = False
```

```
        response: Optional[Dict[str, Any]] = None
        try:
            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
```

```
231                     "play_segments": [],
232                 }
233                 return response
234 
235             segments = result.value if hasattr(result, "value") else result
236             notify("finalizing")
237             _finalize_reingest(video_id, segments, play_wm, cand_wm)
238 
239             response = {
240                 "video_id": video_id,
241                 "status": "success",
242                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
243                 "results": [r.model_dump() for r in val_results],
244                 "play_segments": segments,
245             }
246             return response
247         finally:
248             # Always emit the per-video observability report (next to the video, or to
249             # report.output_dir). Never raises. Surface the paths in the response.
250             paths = emit_indexer_report(
251                 db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
252                 val_results=val_results, val_context=val_context,
253                 segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
254                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
255             )
256             if paths and isinstance(response, dict):
257                 response["reports"] = paths
258 
259 
```

11. assistant (2026-06-21T09:19:54.340Z)

Now I can see the core path. The key observation: `add_video(video_id, req.video_path, ...)` stores `req.video_path` (the original `gs://` URI) as the path column, but `compute_video_id` runs over `local_video` (the fetched local file). Let me examine `acquire_local_input` to confirm the `gs://` routing, and then `add_video`/`get_video` to see if the gs:// path stored breaks the DB row.

12. user (2026-06-21T09:19:55.038Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2 
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5       ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6       today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7       Google Drive (Google Drive for Desktop) as plain files.
8 
9       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12      """
13      from __future__ import annotations
14 
15      import os
16      import tempfile
17      from typing import Any, Dict, Iterator, Optional
18 
19      from backend.storage.base import StorageBackend
```

```
    from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri

    __all__ = [
        "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
        "get_storage", "fetch", "put", "list_uris",
        "acquire_local_input", "input_is_local", "resolve_input_ref",
    ]


    def _storage_dict(storage_config: Any) -> Dict[str, Any]:
        """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
        if storage_config is None:
            return {}
        if hasattr(storage_config, "model_dump"):
            return storage_config.model_dump()  # type: ignore[no-any-return]
        return dict(storage_config)


    def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
        """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
        config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
        callers at the API/CLI boundary catch that and return a clean client error (never a
500).

        Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
        ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
        sc = _storage_dict(storage_config)
        uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
                                input_root=sc.get("input_root", ""))
        return parse_uri(uri)


    def input_is_local(raw: str, storage_config: Any = None) -> bool:
        """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
        plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
        that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
        ``resolve_input_ref``)."""
        return resolve_input_ref(raw, storage_config).backend == "local"


    def acquire_local_input(raw: str, storage_config: Any = None, *,
                            scratch_parent: "str | None" = None) -> str:
        """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.

        Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
        byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
        non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
        path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
        regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
        worker's ``cmd_infer`` already ships."""
        sc = _storage_dict(storage_config)
        ref = resolve_input_ref(raw, sc)
        if ref.backend == "local":
```

```
            return ref.key  # identity ? no temp, no copy, byte-identical to passing the
path directly
        scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
        return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)


    def _backend_class(name: str):
        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
        if name == "local":
            from backend.storage.local import LocalStorage
            return LocalStorage
        if name == "s3":
            from backend.storage.s3 import S3Storage
            return S3Storage
        if name == "gcs":
            from backend.storage.gcs import GCSStorage
            return GCSStorage
        if name == "gdrive":
            from backend.storage.gdrive import GDriveStorage
            return GDriveStorage
        raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")


    def get_storage(backend: Optional[str] = None, *,
                    config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
        """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
        the implementation; its per-backend settings come from ``config[backend]`` merged
with
        ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
        cfg = config or {}
        name = backend or cfg.get("backend", "local")
        settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
        settings.update(overrides)
        return _backend_class(name)(**settings)


    def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
        """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
        (no copy); for s3/gcs it downloads to ``dst``."""
        ref = parse_uri(uri)
        return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)


    def put(local_path: "str | os.PathLike[str]", uri: str, *,
            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
        """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
        ref = parse_uri(uri)
        get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
        return uri


    def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
        """Yield the URI of every object under ``prefix_uri``."""
        ref = parse_uri(prefix_uri)
        for r in get_storage(ref.backend, config=config).list(ref):
            yield r.uri
```

127

13. assistant (2026-06-21T09:19:58.584Z)

Now the critical detail. The claim mentions `gs://` AND `gcs://`. Let me check `parse_uri` and `resolve_input_uri` to see how schemes are mapped ? particularly whether `gs://` is accepted, and whether `gcs://` is accepted. The `_backend_class` only handles `gcs`, not `gs`. Let me read `ref.py`.

14. user (2026-06-21T09:19:59.052Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48      def parse_uri(uri: str) -> StorageRef:
```

```
        """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.

        A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
        have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
        """
        s = str(uri)
        m = _SCHEME_RE.match(s)
        if not m:
            return StorageRef("local", "", s, f"local://{s}")
        scheme = m.group(1).lower()
        rest = s[m.end():]
        if scheme == "gs":
            scheme = "gcs"
        if scheme in ("local", "gdrive"):
            # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
            return StorageRef(scheme, "", rest, s)
        if scheme in _KNOWN:
            bucket, _, key = rest.partition("/")
            return StorageRef(scheme, bucket, key, s)
        raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")


    def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
        """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).

        Precedence (first match wins):
          1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
          2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
          3. A bare/relative name + a non-empty ``input_root`` ? joined under it
             (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
          4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
             ``"gcs://bucket/x"``).
          5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).

        So the default (``input_backend="local"``, ``input_root=""``) is an identity
passthrough for every
        path used today, preserving byte-for-byte behaviour.
        """
        s = str(raw)
        if _SCHEME_RE.match(s):           # 1. explicit scheme wins
            return s
        # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
        #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
        #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
        if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
            return s
        if input_root:                    # 3. join a relative name under the configured
root
            return input_root.rstrip("/") + "/" + s.lstrip("/")
        if input_backend and input_backend != "local":   # 4. prefix a bucket/key with the
declared backend
            return f"{input_backend}://{s}"
        return s                          # 5. bare/relative local path ? unchanged

```

15. assistant (2026-06-21T09:20:03.359Z)

`gs://` is normalized to `gcs://` (ref.py:60-61), and the scheme is accepted. So `gs://` and `gcs://` both route to the `gcs` backend. The fetch path will run `GCSStorage.fetch_to_local`. That part is environmental (network/deps), which I must not count.

Now the critical question for my lens: after a successful fetch, does the DB get written such that `db.get_video(video_id)` returns a row AND the SPA can load it via `?video=`. Let me examine `add_video` and `get_video`, and crucially the SPA's `?video=` query path. Let me read the database methods.

16. user (2026-06-21T09:20:05.190Z)

```
287:    def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
418:    def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
514:    def set_job_video_id(self, job_id: str, video_id: str) -> bool:
520:    def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
```

17. user (2026-06-21T09:20:08.483Z)

```
287         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
288             """Register or update a video row WITHOUT touching its child segments.
289
290             Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
291             with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
292             That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
293             before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
294             row in place; clearing segments is now an explicit, deliberate operation
295             (clear_video_data / prune_segments).
296             """
297             return self._execute_write(
298                 "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
299                 "ON CONFLICT(id) DO UPDATE SET "
300                 "filepath=excluded.filepath, status=excluded.status, "
301                 "validation_results=excluded.validation_results",
302                 (video_id, filepath, status, json.dumps(validation_results)),
303                 "Failed to add video",
304             )
305
306         def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
307             """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
308
309             Captured before a (re)segmentation so the run's new rows (id > watermark) can be
310             told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
311             """
312             with self._connect() as conn:
313                 cur = conn.cursor()
314                 p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
315                                 (video_id,)).fetchone()[0]
316                 c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
317                                 (video_id,)).fetchone()[0]
318                 return int(p), int(c)
319
320         def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
321                            play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
```

```
                            cand_after: Optional[int] = None) -> None:
            """Delete play/candidate segments by id range (events cascade from
play_segments).

            Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
            successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
            and keep the OLD ones when the run failed/produced nothing (``*_after``).
            """
            with self._connect() as conn:
                cur = conn.cursor()
                if play_upto is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
                if play_after is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
                if cand_upto is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
                if cand_after is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
                conn.commit()

        def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
                return None

        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
            return self._execute_write(
                "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
                (segment_id, timestamp, event_type, player, json.dumps(details or {})),
                "Failed to add event",
            )

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
```

```
371                     cursor.execute(
372                         """INSERT INTO candidate_segments
373                            (video_id, detector, chunk_index, start_time, end_time, duration,
374                             confidence, stats, detection_params)
375                            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
376                         (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
```

18. assistant (2026-06-21T09:20:13.238Z)

`add_video` stores `filepath = req.video_path` (the `gs://` URI), but that's just a text column ? it doesn't break `get_video(video_id)`. The video_id is what's keyed. Now let me read `get_video` to confirm the row returns regardless of the filepath value.

19. user (2026-06-21T09:20:14.430Z)

```
418         def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
419             with self._connect() as conn:
420                 row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
421                 if row:
422                     d = dict(row)
423                     d["validation_results"] = json.loads(d["validation_results"] or "[]")
424                     return d
425             return None
426 
427         def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
428             """
429             Dynamically queries play segments based on filters:
430             - duration_min: float
431             - server: string
432             - receiver: string
433             - event_type: string (e.g. smash, foul)
434             """
435             query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
436             params = [video_id]
437 
438             if filters.get("duration_min"):
439                 query += " AND r.duration >= ?"
440                 params.append(filters["duration_min"])
441 
442             if filters.get("server"):
443                 query += " AND r.server = ?"
444                 params.append(filters["server"])
445 
446             if filters.get("receiver"):
447                 query += " AND r.receiver = ?"
448                 params.append(filters["receiver"])
449 
450             if filters.get("event_type"):
451                 # Check if there is an event of a specific type inside this segment
452                 query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
453                 params.append(filters["event_type"])
454 
455             segments_list = []
456             with self._connect() as conn:
457                 cursor = conn.cursor()
458                 rows = cursor.execute(query, params).fetchall()
459 
460                 for row in rows:
461                     r_dict = dict(row)
462                     r_id = r_dict["id"]
```

20. assistant (2026-06-21T09:20:20.812Z)

`get_video(video_id)` keys purely on `id`, so the stored `gs://` filepath does not prevent the row from being returned. The DB write (`add_video`) clearly persists with `video_id` as the key. This strongly supports the claim for the worker/DB-write path.

Now the SPA-loadability via `?video=`. The claim says "the SPA can load via ?video=". Let me find the endpoint the SPA hits and whether it could diverge for gs:// origin videos. Let me check how the SPA serves the actual video file ? this is where a `gs://` path could break: the SPA needs to *play* the video, and if it tries to serve the file from the `filepath` column (now a `gs://` URI), playback would fail. But the claim is specifically about "load via ?video=" and "db.get_video() returns" ? let me check what `?video=` actually triggers.

21. user (2026-06-21T09:20:22.885Z)

No files found

22. user (2026-06-21T09:20:23.950Z)

```
25:from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
54:@app.get("/", response_class=HTMLResponse)
123:@app.get("/api/config")
127:@app.get("/api/health")
260:@app.post("/api/process", status_code=202)
303:@app.get("/api/jobs/{job_id}")
336:@app.get("/api/highlights/{video_id}/{filename}")
356:    media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
357:    return FileResponse(path, media_type=media, filename=name)
360:@app.post("/api/query")
371:@app.post("/api/compile")
```

23. assistant (2026-06-21T09:20:26.218Z)

Let me find the SPA HTML and the `?video=` query handling. Let me look at the root endpoint and frontend.

24. user (2026-06-21T09:20:27.678Z)

<tool_use_error>Path does not exist: C:/Users/avidu/Projects/badminton-highlight-indexer/frontend. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.</tool_use_error>

25. user (2026-06-21T09:20:27.938Z)

```
backend\api\static\index.html
docs\mockups\ui_alpha_mockup.html
.claude\worktrees\bhi-execpolicy\backend\api\static\index.html
.claude\worktrees\bhi-execpolicy\docs\mockups\ui_alpha_mockup.html
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\api\static\index.html
.claude\worktrees\agent-a771ff1d85e18f2ed\docs\mockups\ui_alpha_mockup.html
.claude\worktrees\tier1-windowing\backend\api\static\index.html
.claude\worktrees\tier1-windowing\docs\mockups\ui_alpha_mockup.html
.claude\worktrees\w2-safeguard\backend\api\static\index.html
.claude\worktrees\w2-safeguard\docs\mockups\ui_alpha_mockup.html
.claude\worktrees\hardchop\backend\api\static\index.html
.claude\worktrees\hardchop\docs\mockups\ui_alpha_mockup.html
.claude\worktrees\boxhill17\backend\api\static\index.html
.claude\worktrees\boxhill17\docs\mockups\ui_alpha_mockup.html
.claude\worktrees\pr-stitch\backend\api\static\index.html
.claude\worktrees\pr-stitch\docs\mockups\ui_alpha_mockup.html
.claude\worktrees\gap-doc\backend\api\static\index.html
.claude\worktrees\gap-doc\docs\mockups\ui_alpha_mockup.html
.claude\worktrees\resweep\backend\api\static\index.html
.claude\worktrees\resweep\docs\mockups\ui_alpha_mockup.html
```

```
.claude\worktrees\recdoc\backend\api\static\index.html
.claude\worktrees\recdoc\docs\mockups\ui_alpha_mockup.html
artifacts\landing\capture.html
artifacts\site_deploy\capture.html
artifacts\site_deploy\index.html
artifacts\landing\index.html
```

26. user (2026-06-21T09:20:32.633Z)

```
15:                <p>Local AI-powered video validation, segmentation, and clip indexing</p>
28:                <div class="glass-panel" id="video-input-panel">
34:                        <label for="video-path-input">Local Video File Path</label>
35:                        <input type="text" id="video-path-input" class="text-input"
placeholder="e.g. c:/videos/badminton_gopro.mp4" value="">
41:                    <button id="btn-process-video" class="btn-primary">
55:                            <p style="font-size: 0.85rem;">Load a video to trigger the
pluggable validation checklist.</p>
110:                            <p>Enter a local video filepath in the sidebar and run indexing
to build the database index.</p>
124:                            <input type="text" id="highlight-output-name" class="text-input"
style="padding: 8px 12px; font-size: 0.85rem; width: 220px;" value="highlights_compiled.mp4">
126:                        <button id="btn-compile-highlights" class="btn-primary">
```

27. assistant (2026-06-21T09:20:35.587Z)

The static `index.html` references "Local Video File Path" but doesn't seem to use `?video=`. Let me search for the `?video=` query-param loading specifically across the JS in this file and the static dir.

28. user (2026-06-21T09:20:36.724Z)

```
backend\api\static\app.js:125:            currentVideoId = data.video_id;
backend\api\static\app.js:333:                    video_id: currentVideoId,
backend\api\static\app.js:377:                    video_id: currentVideoId,
```

29. assistant (2026-06-21T09:20:41.982Z)

There's no `?video=` query-param mechanism in the actual SPA. But my lens is specifically the worker/execution + DB write path, not the SPA wiring. The claim's SPA-loadability is a separate lens. For MY lens, the question is narrowly: does the worker path write a `video_id` that `db.get_video(video_id)` returns. Let me confirm the remaining pieces: `set_job_video_id`, `mark_job_done`, and the `/api/jobs/{job_id}` endpoint that surfaces `video_id` to the client. Let me also verify `add_video`'s `_execute_write` actually commits.

30. user (2026-06-21T09:20:42.886Z)

```
505                f"Failed to mark job {job_id} running",
506            )
507
508        def set_job_progress(self, job_id: str, progress: str) -> bool:
509            return self._execute_write(
510                "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
511                f"Failed to set progress for job {job_id}",
512            )
513
514        def set_job_video_id(self, job_id: str, video_id: str) -> bool:
515            return self._execute_write(
516                "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
517                f"Failed to set video_id for job {job_id}",
518            )
519
520        def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
521            """Job EXECUTED to completion. The pipeline's own verdict (success/failed
522            validation) is inside `result` ? job status 'done' means 'the worker
```

```
finished'."""
523             return self._execute_write(
524                 "UPDATE jobs SET status='done', progress='finished', result=?, "
525                 "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
526                 (json.dumps(result), job_id),
527                 f"Failed to mark job {job_id} done",
528             )
529
530         def mark_job_failed(self, job_id: str, error: str) -> bool:
531             """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
532             return self._execute_write(
533                 "UPDATE jobs SET status='failed', error=?, "
534                 "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
535                 f"Failed to mark job {job_id} failed",
536             )
537
538         def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
539             """Fetch one job; `result`/`player_pool` JSON deserialized."""
540             with self._connect() as conn:
541                 row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
542                 if not row:
543                     return None
544                 d = dict(row)
545                 d["result"] = json.loads(d["result"]) if d["result"] else None
546                 d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
547                 d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
548                 return d
549
550         # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
-------------------------
551         def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
552             """Attach/replace a job's JSON ExecutionPolicy."""
553             return self._execute_write(
554                 "UPDATE jobs SET policy=? WHERE id = ?",
555                 (json.dumps(policy) if policy is not None else None, job_id),
556                 f"Failed to set policy for job {job_id}",
557             )
558
559         def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
560             """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
561             worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
562             computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
563             return self._execute_write(
564                 "UPDATE jobs SET status='waiting', "
```

31. assistant (2026-06-21T09:20:47.954Z)

The DB-write path is solid: on a successful (or even validation-failed) run, `add_video(video_id, ...)` is always called, persisting a row keyed by `video_id`, and `get_video(video_id)` returns it. The worker also calls `set_job_video_id` so the job row carries `video_id` (surfaced via `/api/jobs/{job_id}`).

Now let me verify the draining actually happens under default single-user config ? the claim depends on `_drain_due_jobs` being invoked after the 202. If the worker never drains by default, the job never executes and no `video_id` is ever written. Let me read the `/api/process` endpoint and how draining is triggered.

32. user (2026-06-21T09:20:49.032Z)

```
    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
            # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
            input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
        # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
        # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
        # rejects a non-local URI as out-of-root.
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_drain_due_jobs)
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}")
    def get_job(job_id: str):
        """Job state: {status, progress, result, reports}. `status` = execution state
        (queued|running|done|failed); the pipeline verdict is result["status"]."""
        job = db_instance.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        result = job.get("result")
        return {
            "job_id": job["id"],
            "status": job["status"],
            "progress": job.get("progress"),
```

```
315             "video_id": job.get("video_id"),
316             "error": job.get("error"),
317             "result": result,
318             "reports": (result or {}).get("reports"),
319             "created_at": job.get("created_at"),
320             "started_at": job.get("started_at"),
321             "finished_at": job.get("finished_at"),
322         }
323
324     # --- Chunked upload (B2, PR-2) ----------------------------------------------------
325     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
326     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
327     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
328     # sequential-part logic, and abort cleanup ? is unchanged.
329     from backend.api import uploads as uploads_api
330
331     app.include_router(uploads_api.router)
332
333
334     # --- Highlight download (B3, PR-2) ------------------------------------------------
```

33. assistant (2026-06-21T09:20:54.520Z)

Critical: line 296 `_get_executor().submit(_drain_due_jobs)` is called right after the 202, so the worker DOES drain by default. Confirmed.

Now there's one important subtlety for my lens. At line 274, the submit-time path calls `resolve_input_ref` (not a fetch ? cheap, no network). For `gs://`, this returns a `gcs` backend ref, so `input_ref.backend == "local"` is False ? the local-existence check at line 283 is skipped ? job is created. Good, the gs:// case passes submit. But I must check `validate_input_path` ? does it reject a `gs://` URI under DEFAULT config (no `allowed_video_root`)? Let me read it.

34. user (2026-06-21T09:20:55.696Z)

backend\utils\paths.py:53:def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:

35. user (2026-06-21T09:20:58.511Z)

```
53      def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
54          """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
55
56          With no allowed_root (default, single-user local mode) only the null-byte check
applies,
57          so the local 'point it at any of my videos' workflow is preserved. Setting an
58          allowed_root (a future hosted/tenant deployment) enforces containment.
59          """
60          if "\x00" in path:
61              raise ValueError("video_path must not contain a null byte")
62          if allowed_root:
63              resolve_under_root(path, allowed_root)
64
```

36. assistant (2026-06-21T09:21:02.874Z)

Under default single-user config (`allowed_video_root=None`), `validate_input_path` only checks for null bytes. A `gs://` URI passes. So submit succeeds, job is queued, worker drains.

Now let me close the loop on the actual fetch path for `gcs` to confirm there's no *code* defect (vs environmental dep/auth). The `fetch` calls `GCSStorage.fetch_to_local`. Let me read `gcs.py` to ensure that, given the SDK and auth are available (environmental, excluded), the code routes

correctly and returns a local path.

37. user (2026-06-21T09:21:03.337Z)

```
1       """Google Cloud Storage backend ? first-class object buckets.
2
3       Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4       install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5       (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6       never from config. A ``client`` may be injected for tests.
7       """
8       from __future__ import annotations
9
10      from datetime import timedelta
11      from typing import Any, Iterator, Optional
12
13      from backend.storage.base import StorageBackend
14      from backend.storage.ref import StorageRef, StorageStat
15
16
17      class GCSStorage(StorageBackend):
18          def __init__(self, *, client: Any = None, project: Optional[str] = None,
19                       signed_url_ttl: int = 3600, **_ignored) -> None:
20              self._ttl = signed_url_ttl
21              if client is not None:
22                  self._client = client
23                  return
24              from google.cloud import storage  # type: ignore  # lazy: only when a gcs:// URI
is used
25              self._client = storage.Client(project=project)
26
27          def _blob(self, ref: StorageRef):
28              return self._client.bucket(ref.bucket).blob(ref.key)
29
30          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31              if dst is None:
32                  raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33              import os
34              dst = str(dst)
35              if os.path.exists(dst) and not overwrite:
36                  return dst
37              os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38              self._blob(ref).download_to_filename(dst)
39              return dst
40
41          def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42              self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43              return ref
44
45          def exists(self, ref) -> bool:
46              return bool(self._blob(ref).exists())
47
48          def stat(self, ref) -> StorageStat:
49              b = self._blob(ref)
50              b.reload()
51              return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                                 content_type=b.content_type)
53
54          def list(self, prefix) -> Iterator[StorageRef]:
55              for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56                  yield StorageRef("gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")
```

```
57
58          def delete(self, ref) -> None:
59              self._blob(ref).delete()
60
61          def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
62              return self._blob(ref).generate_signed_url(
63                  version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)
64
```

38. assistant (2026-06-21T09:21:08.981Z)

`fetch_to_local` downloads to `dst` and returns the local path. The `dst` is computed in `acquire_local_input` as `os.path.join(scratch, ref.name or "in.mp4")`. For `gs://bucket/videos/x.mp4`, `ref.key = "videos/x.mp4"`, and `ref.name` = `"x.mp4"` (via the `name` property rsplit). So `dst` is a valid local path. No code defect in routing.

Now let me check one more potential divergence in my lens: `compute_video_id` on the fetched local file. Let me verify it hashes the file content (not the path), so the gs:// case yields a valid id.

39. user (2026-06-21T09:21:09.852Z)

```
1       """Canonical file-identity hashing for the indexer.
2
3       `video_id` is the MD5 of the *whole* video file. Historically each entrypoint
4       (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
`scripts/run_process_and_stitch.py`)
5       defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
6       The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
7       any future change to one copy would silently diverge the DB key.
8
9       This module is the single source of truth. Import `compute_video_id` everywhere a
10      video needs to be keyed.
11      """
12
13      from __future__ import annotations
14
15      import hashlib
16
17      # 64 KB matches the value used by the live pipeline (backend/main.py) before this
18      # was centralized. The chunk size does not affect the resulting digest ? it only
19      # bounds memory use while streaming large files.
20      _CHUNK_SIZE = 65536
21
22
23      def compute_video_id(filepath: str) -> str:
24          """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26          Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27          into memory at once.
28          """
29          hasher = hashlib.md5()
30          with open(filepath, "rb") as f:
31              for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
32                  hasher.update(chunk)
33          return hasher.hexdigest()
34
```

40. assistant (2026-06-21T09:21:18.085Z)

`compute_video_id` opens and hashes the *local* file path returned by `acquire_local_input`. For the gs:// case, that's the fetched scratch file. Hashing works identically to the local case. No divergence.

Let me do a final check on the default config to confirm `input_backend`/`input_root` defaults don't interfere, and that `global_config.storage` exists by default (so `acquire_local_input` gets sane defaults). Let me check the StorageConfig defaults.

41. user (2026-06-21T09:21:20.302Z)

```
297:class StorageConfig(BaseModel):
310:    # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
311:    # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
315:    input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
316:    input_root: str = ""
```

42. assistant (2026-06-21T09:21:41.568Z)

Defaults: `input_backend="local"`, `input_root=""`. With an explicit `gs://` scheme on `req.video_path`, `resolve_input_uri` precedence rule #1 (explicit scheme wins, ref.py:87-88) returns the URI as-is regardless of these defaults. So `gs://...` routes to the gcs backend under default config. Confirmed ? no interference.

Let me verify the full chain one final time for the divergence question, specifically the `finally` block at main.py:247-257 ? `emit_indexer_report` is called with `req.video_path` (the gs:// URI). If that raises... but it's documented "Never raises." And even if it did, it's in `finally` after `response` is set and `add_video` already committed. Let me confirm `_finalize_reingest` persists segments and that nothing in the gs:// path diverges there. Actually, segments are written by `segmenter.process_video` via `add_play_segment` ? that's detector logic identical for both local and gs:// (it operates on `local_video` + `video_id`). No divergence.

I have thoroughly traced the worker/execution + DB-write path. Let me record my analysis.

The chain under default single-user config for a `gs://` (or `gcs://`) input:

1. Submit (`/api/process`, main.py:260-300): `validate_input_path` passes (null-byte only, no `allowed_video_root` default); `resolve_input_ref` returns a `gcs` backend ref (gs:// normalized to gcs:// at ref.py:60-61); local-existence check skipped because `backend != "local"` (main.py:283); job created `queued`; `_get_executor().submit(_drain_due_jobs)` fires (main.py:296).
2. Drain (job_worker.py:100-121): `claim_due_job` ? `_run_claimed_job` ? `_job_to_request` rebuilds `ProcessRequest` from the row's `video_path` (the gs:// URI, stored verbatim).
3. `_execute_processing` (main.py:140-258): `acquire_local_input(req.video_path, ...)` ? resolves to gcs ref ? `tempfile.mkdtemp` + `fetch(...)` ? `GCSStorage.fetch_to_local` downloads to scratch and returns a local path (gcs.py:30-39). `compute_video_id(local_video)` hashes the fetched bytes (hashing.py). `add_video(video_id, req.video_path, ...)` UPSERTs a row keyed by `video_id` ? always called on the validation-fail branch (179), the processed branch (190), even though it stores the gs:// URI in the `filepath` text column (harmless to keying).
4. `db.get_video(video_id)` (database.py:418-425) selects `WHERE id = ?` ? returns the row regardless of the filepath column value.
5. Worker also persists `set_job_video_id` + `mark_job_done`, surfaced via `/api/jobs/{job_id}` ? `video_id`.

No place in the worker/DB-write path diverges between the gs:// and local case other than the fetch-to-scratch step (which is environmental: SDK + ADC), explicitly excluded. The DB write is keyed on the content hash, identical for both. I found NO concrete code-path defect that fails to yield a queryable API-DB `video_id` for a valid gs:// input under default config.

One caveat worth noting (not a refutation under my lens): the actual SPA in `backend/api/static/` has no `?video=` query-param loader (`app.js` has no `URLSearchParams`/`location.search`), so the "SPA can load via ?video=" sub-claim is unsupported by the shipped frontend. But that is the SPA/frontend lens, not the worker/DB-write lens I was assigned, and it does not affect whether `db.get_video(video_id)` returns a queryable row ? which it does.

43. user (2026-06-21T09:21:55.289Z)

Structured output provided successfully

44. assistant (2026-06-21T09:22:05.446Z)

I completed the adversarial analysis of the worker/execution + DB-write path and returned my structured verdict.

Summary: I could NOT refute the claim within my lens (refuted=false). Under default single-user config, a valid `gs://`|`gcs://` URI flows cleanly through the post-202 worker path and produces a `video_id` that `db.get_video(video_id)` returns:

- `backend/main.py:269-296` ? submit accepts gs:// (null-byte-only validation, gcs-backend ref, local-existence check skipped), then fires `_drain_due_jobs`.
- `backend/api/job_worker.py:84-116` ? drain rebuilds the request and runs it identically to a local path.
- `backend/main.py:159-160` ? `acquire_local_input` fetches the gcs object to scratch (`backend/storage/gcs.py:30-39`) and `compute_video_id` hashes the fetched bytes.
- `backend/main.py:179,190` + `backend/infrastructure/database.py:287-304` ? `add_video` always UPSERTs a row keyed on the content hash; `get_video` (database.py:418-425) returns it `WHERE id=?` regardless of the stored gs:// filepath.

The only gs://-vs-local divergence is the network fetch itself (missing `google-cloud-storage` dep / missing ADC), which is environmental and explicitly excluded. The DB write is hash-keyed and byte-for-byte identical for both cases, so no concrete code-path counterexample exists.

One out-of-lens caveat I flagged: the shipped SPA (`backend/api/static/app.js`) has no `?video=` query-param loader, so the claim's "SPA can load via ?video=" sub-clause is unsupported by the actual frontend ? but that belongs to the frontend lens and does not affect the queryable-DB-row conclusion for my lens.